

Arquitectura de Datos

Especialización en Internet de las Cosas

Docente:

Lacheski, Martín Anibal

Clase 3:

Bases de Datos Relacionales y Lenguaje SQL



Arquitectura de Datos

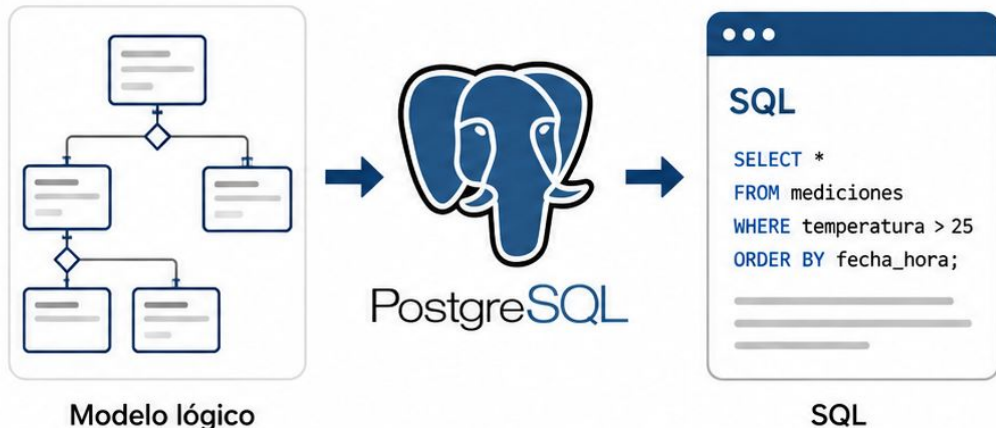
Del modelo a la implementación

En la clase anterior terminamos con una estructura lógica capaz de representar nuestro sistema.

Ya decidimos:

- qué información necesitamos;
- cómo identificarla;
- cómo relacionarla;
- qué reglas debe respetar.

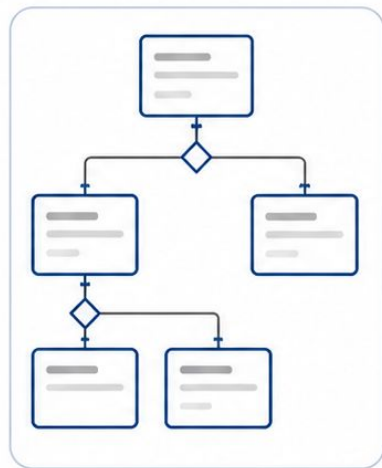
Ahora vamos a implementar esa estructura y comenzar a utilizar sus datos.



El modelo nos dice qué representar.
SQL nos permitirá implementarlo y consultarlo.

Hasta ahora diseñamos

Hoy empezamos a operar sobre los datos



Clase 2

Modelo lógico



Clase 3

Crear estructuras



Insertar datos



Consultar



Modificar



Relacionar y analizar



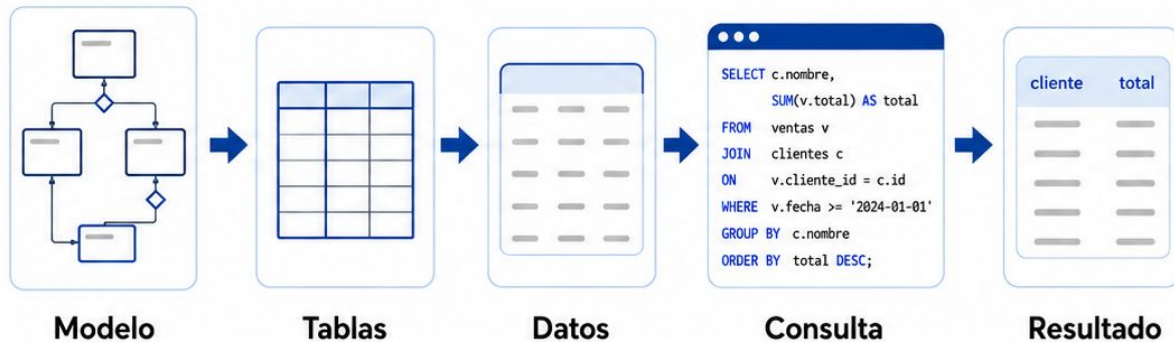
Pasamos de decidir cómo organizar los datos a trabajar efectivamente con ellos.

Objetivos de la clase

Implementar y consultar un modelo relacional

Vamos a aprender a:

- representar el modelo mediante tablas;
- implementar claves y restricciones;
- crear estructuras con SQL;
- insertar, modificar y eliminar datos;
- realizar consultas;
- filtrar y ordenar resultados;
- calcular métricas;
- combinar información relacionada;
- construir consultas de mayor complejidad;
- ejecutar operaciones básicas de forma controlada.



Una base relacional se vuelve útil cuando podemos mantener y consultar correctamente la información que contiene.

Monitoreo de ambientes

Seguimos con el mismo sistema

Vamos a trabajar principalmente con:

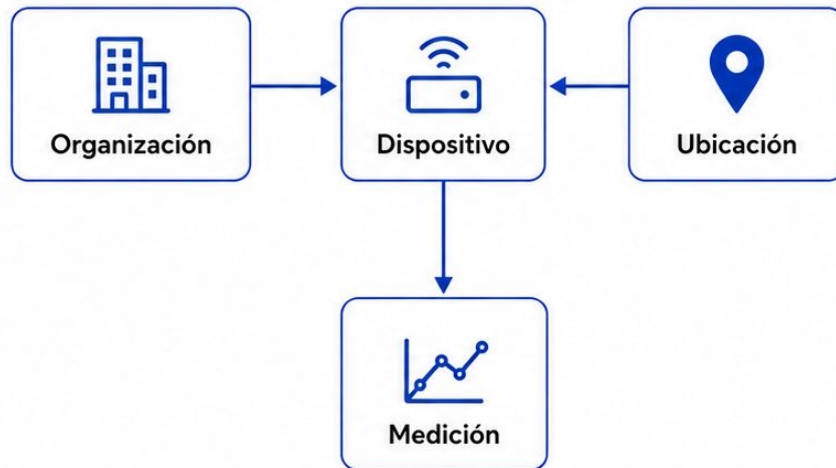
 **Organización**

 **Ubicación**

 **Dispositivo**

 **Medición**

- Una organización administra dispositivos.
- Un dispositivo puede estar instalado en una ubicación.
- Cada dispositivo genera mediciones ambientales a lo largo del tiempo.

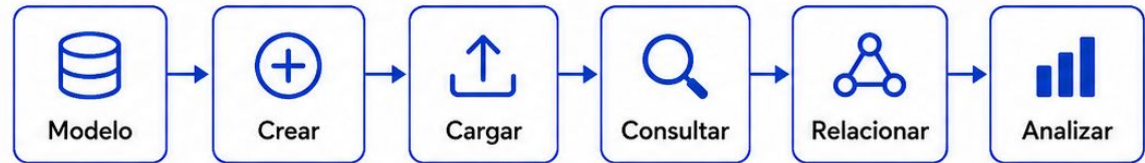


**No diseñaremos nuevamente el modelo.
Vamos a utilizar el que ya construimos.**

Arquitectura de Datos

Agenda del día

- Del modelo lógico al modelo relacional.
- Claves e integridad.
- SQL: DDL, DML y DQL.
- Creación de tablas.
- Inserción, actualización y eliminación.
- SELECT, filtros y ordenamiento.
- Agregaciones y agrupamientos.
- JOIN.
- Subconsultas y CTE.
- Introducción a funciones de ventana.
- Transacciones.
- Consultas parametrizadas.
- Práctica guiada.



Vamos a recorrer el camino completo desde la estructura hasta la información que necesita una aplicación.

1

Del Modelo Lógico al Modelo Relacional



id_dispositivo	nombre	id_organizacion	id_ubicacion
1	Sensor A1	1	2
2	Gateway B1	1	3
3	Sensor C2	2	1
4	Sensor D3	2	4
...	...	...	...

El modelo ya existe

Ahora debemos implementarlo

Las decisiones de diseño comienzan a tomar forma

Modelo lógico:



Modelo relacional:

dispositivos

dispositivo_id	organizacion_id	ubicacion_id	nombre	numero_serie
_____	_____	_____	_____	_____
_____	_____	_____	_____	_____
_____	_____	_____	_____	_____



No estamos descubriendo una nueva entidad.

Estamos implementando una decisión tomada anteriormente.

Relaciones y tablas

La estructura que utilizaremos

En la implementación trabajaremos con:

Tabla

Representa una relación del modelo.

Fila

Representa una instancia almacenada.

Columna

Representa una propiedad dentro de esa estructura.

Ejemplo: `dispositivos`

contiene muchas filas, cada una correspondiente a un dispositivo.

dispositivos

Columnas				← Fila
dispositivo_id	organizacion_id	ubicacion_id	nombre	
—	—	—	—	
101	7	3	Sensor A	
—	—	—	—	



El esquema define la estructura.
Las filas contienen los datos concretos.

¿Qué datos admite una columna?

El dominio también forma parte de la estructura

nombre	→	texto
activo	→	verdadero / falso
valor	→	número
timestamp	→	fecha y hora

Cada columna debe admitir valores compatibles con lo que representa.

nombre Aa _____ _____ _____	activo ☒ verdadero ☐ falso _____ _____
valor 123 42 3.14 -7	timestamp  2025-05-20 10:30:00 2024-12-01 08:15:45



Elegir un tipo de dato también significa limitar qué valores pueden almacenarse.

Dos cosas diferentes

La estructura y su contenido

Esquema

dispositivo_id	nombre	numero_serie



Datos

dispositivo_id	nombre	numero_serie
101	Sensor Aula 1	SN-A001
102	Sensor Aula 2	SN-A002
103	Sensor Lab 1	SN-B014



dispositivo_id
→ número



nombre
→ texto



numero_serie
→ texto



fecha_registro
→ fecha y hora

Cada columna debe admitir valores compatibles con lo que representa.



Elegir un tipo de dato
también significa limitar qué valores pueden almacenarse.

2

Claves e Integridad

organizaciones

organizacion_id PK	nombre	activo
1	Escuela Técnica	true
2	Universidad	true
3	Empresa Solar	true

dispositivos

dispositivo_id PK	organizacion_id FK	nombre	numero_serie
101	1	Sensor Aula 1	SN-A001
102	1	Sensor Aula 2	SN-A002
201	2	Sensor Lab 1	SN-B001
301	3	Sensor Panel	SN-C001



Clave primaria

Implementar la identidad de cada fila

En el modelo ya decidimos utilizar:

dispositivo_id

En la tabla indicaremos:

dispositivo_id BIGINT PRIMARY KEY

La clave primaria:

- identifica una fila;
- no puede repetirse;
- no puede ser nula.

Tabla: dispositivos

dispositivo_id PK	organizacion_id FK	nombre	numero_serie
101	1	Sensor Aula 1	SN-A001
102	1	Sensor Aula 2	SN-A002
201	2	Sensor Lab 1	SN-B001
301	3	Sensor Panel	SN-C001



La PRIMARY KEY implementa la identidad que definimos durante el modelado.

Clave foránea

Implementar relaciones entre tablas

Un dispositivo pertenece a una organización.

En dispositivos podemos guardar:

organization_id BIGINT

y declarar:

```
FOREIGN KEY (organization_id)
REFERENCES organizaciones(organization_id)
```

organizaciones

organization_id PK	nombre
1	Escuela Técnica
2	Universidad
3	Empresa Solar

PK
organizaciones.organization_id

FK
dispositivos.organization_id

dispositivos

dispositivo_id PK	organization_id FK	nombre	numero_serie
101	1	Sensor Aula 1	SN-A001
102	1	Sensor Aula 2	SN-A002
201	2	Sensor Lab 1	SN-B001
301	3	Sensor Panel	SN-C001



Una FOREIGN KEY conecta filas de tablas relacionadas.

Una referencia debe ser válida

Evitar relaciones inexistentes

Si intentamos registrar:

```
dispositivo_id = 101  
organizacion_id = 999
```

pero la organización 999 no existe,
la operación debería rechazarse.



Referencia válida

organizaciones

organizacion_id PK	nombre
1	Escuela Técnica
2	Universidad
3	Empresa Solar

dispositivos

dispositivo_id PK	organizacion_id FK	nombre
101	1	Sensor Aula 1
102	2	Sensor Aula 2
201	3	Sensor Lab 1



Referencia inválida

organizaciones

organizacion_id PK	nombre
1	Escuela Técnica
2	Universidad
3	Empresa Solar

dispositivos

dispositivo_id PK	organizacion_id FK	nombre
101	999	Sensor Aula 1
102	2	Sensor Aula 2



La organización 999 no existe.
La operación debe rechazarse.



La integridad referencial evita que una relación
apunte a algo que no existe.

Las reglas llegan a la base

Implementar decisiones del modelo



Número de serie obligatorio:

numero_serie TEXT NOT NULL



Número de serie único:

numero_serie TEXT UNIQUE



Porcentaje válido:

CHECK (bateria BETWEEN 0 AND 100)

Tabla: dispositivos

dispositivo_id PK	organizacion_id FK	nombre	numero_serie UNIQUE	bateria CHECK 0-100
101	1	Sensor Aula 1	SN-A001	87
102	1	Sensor Aula 2	SN-A002	65
201	2	Sensor Lab 1	SN-B001	100
301	3	Sensor Panel	SN-C001	42
401	2	Sensor Inválido 1	NULL	75
402	2	Sensor Inválido 2	SN-A001	30
403	2	Sensor Inválido 3	SN-D001	150

✗ Falla NOT NULL

✗ Falla UNIQUE

✗ Falla CHECK



Las restricciones que definimos conceptualmente
pueden convertirse en reglas verificables por el SGBD.

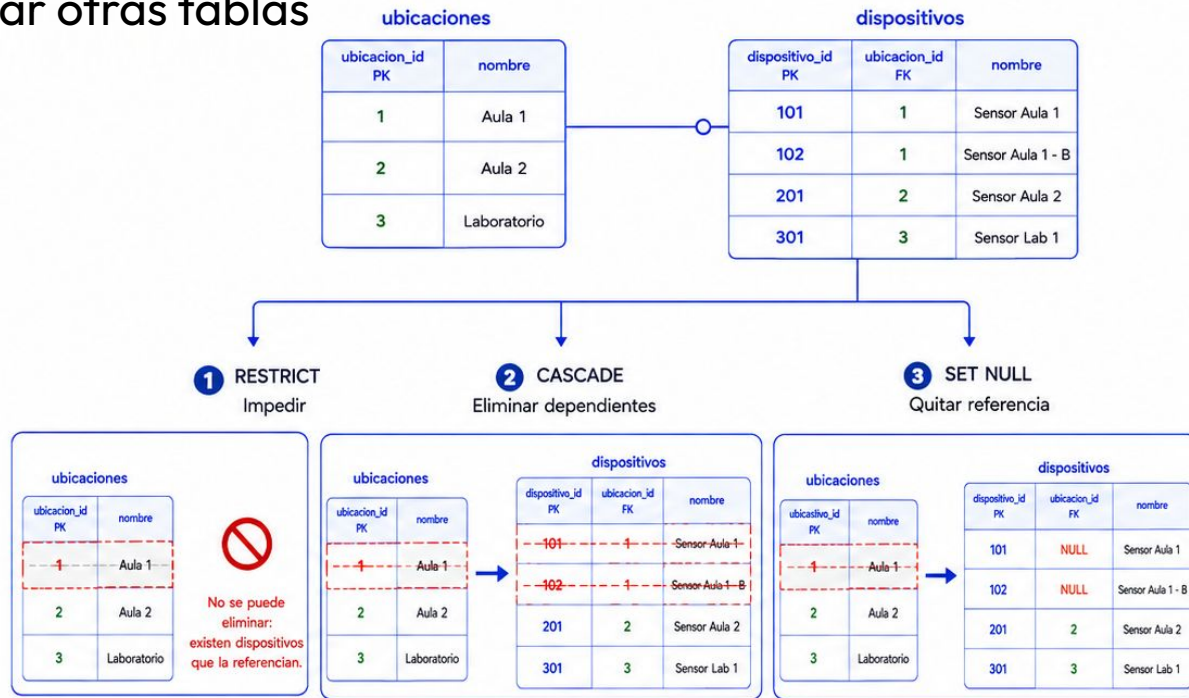
Relaciones y eliminación

Eliminar una fila puede afectar otras tablas

Supongamos que una ubicación todavía posee dispositivos asociados.
¿Qué debería ocurrir si intentamos eliminarla?

Algunas estrategias posibles son:

- impedir la eliminación;
- eliminar también información dependiente;
- quitar la referencia cuando el modelo lo permita.



Las relaciones también condicionan qué operaciones pueden realizarse de manera segura.

3

SQL y Definición de Estructuras



```
CREATE TABLE organizaciones (  
  organizacion_id INT PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  activo BOOLEAN NOT NULL  
);  
  
CREATE TABLE dispositivos (  
  dispositivo_id INT PRIMARY KEY,  
  organizacion_id INT NOT NULL,  
  nombre VARCHAR(100) NOT NULL,  
  numero_serie VARCHAR(50) NOT NULL,  
  FOREIGN KEY (organizacion_id)  
    REFERENCES organizaciones(organizacion_id)  
);
```



organizaciones

organizacion_id (PK)	nombre	activo
1	Escuela Técnica	true
2	Universidad	true
3	Empresa Solar	true
...	...	...

dispositivos

dispositivo_id (PK)	organizacion_id (FK)	nombre	numero_serie
101	1	Sensor Aula 1	SN-A001
102	1	Sensor Aula 2	SN-A002
201	2	Sensor Lab 1	SN-B001
301	3	Sensor Panel	SN-C001
...	...	...	...

Structured Query Language

Un lenguaje para trabajar con bases relacionales

Durante la clase utilizaremos SQL para tres tipos principales de operaciones:

DDL

Definir estructuras.

DML

Modificar datos.

DQL

Consultar datos.



SQL permite expresar qué queremos hacer con la información almacenada.

Data Definition Language

Definir la estructura

Algunas instrucciones DDL:

- CREATE TABLE
- ALTER TABLE
- DROP TABLE

Hoy nos concentraremos principalmente en crear el esquema inicial.

SQL

```
CREATE TABLE organizaciones (  
  organizacion_id BIGINT PRIMARY KEY,  
  nombre TEXT NOT NULL  
);  
  
CREATE TABLE dispositivos (  
  dispositivo_id BIGINT PRIMARY KEY,  
  organizacion_id BIGINT  
    REFERENCES organizaciones  
    (organizacion_id),  
  nombre TEXT NOT NULL,  
  numero_serie TEXT UNIQUE  
);
```

Tabla: organizaciones

organizacion_id PK	nombre
1	Escuela Técnica
2	Universidad
3	Empresa Solar



Tabla: dispositivos

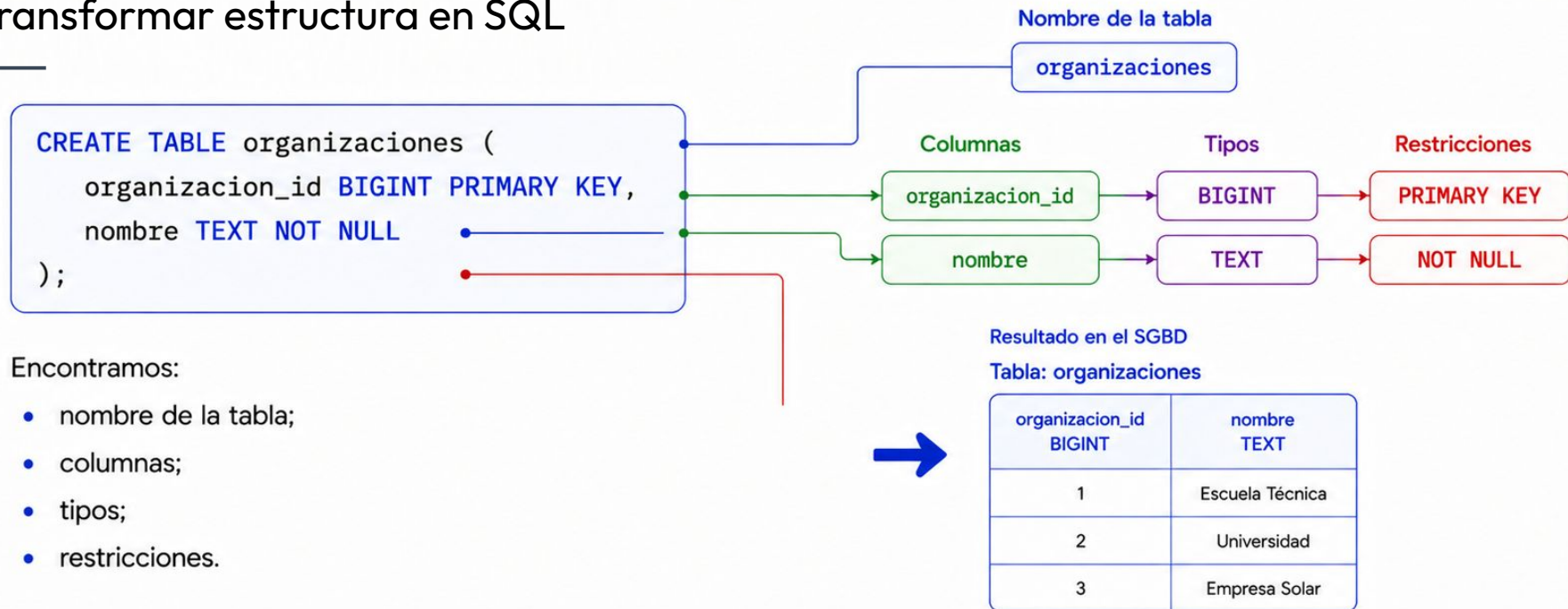
dispositivo_id PK	organizacion_id FK	nombre	numero_serie UNIQUE
101	1	Sensor Aula 1	SN-A001
102	1	Sensor Aula 2	SN-A002
201	2	Sensor Lab 1	SN-B001
301	3	Sensor Panel	SN-C001



DDL trabaja sobre la estructura de la base,
no sobre las filas individuales.

Crear una tabla

Transformar estructura en SQL



Encontramos:

- nombre de la tabla;
- columnas;
- tipos;
- restricciones.



CREATE TABLE materializa una parte del modelo dentro del SGBD.

Identificadores automáticos

La base puede generar la clave

```
organizacion_id BIGINT  
GENERATED ALWAYS AS IDENTITY  
PRIMARY KEY
```

Al insertar una organización no
necesitamos elegir manualmente
su identificador.

Tabla: organizaciones

organizacion_id (GENERATED)	nombre TEXT
1	Escuela Técnica
2	Universidad
3	Empresa Solar
4	Municipalidad



INSERT



```
INSERT INTO organizaciones  
(nombre) VALUES ('Empresa Solar');
```



La base genera
el identificador



Una clave interna puede ser generada
automáticamente por la base de datos.

Incorporar relaciones

La tabla comienza a conectarse con otras

```
CREATE TABLE dispositivos (  
    dispositivo_id BIGINT  
        GENERATED ALWAYS AS IDENTITY  
        PRIMARY KEY,  
    organization_id BIGINT NOT NULL,  
    ubicacion_id BIGINT,  
    nombre TEXT NOT NULL,  
    numero_serie TEXT NOT NULL UNIQUE,  
  
    FOREIGN KEY (organization_id)  
        REFERENCES organizaciones(organization_id),  
  
    FOREIGN KEY (ubicacion_id)  
        REFERENCES ubicaciones(ubicacion_id)  
);
```

Tabla: organizaciones

organization_id (PK)	nombre
1	Escuela Técnica
2	Universidad
3	Empresa Solar

Tabla: ubicaciones

ubicacion_id (PK)	organization_id (FK)	nombre
1	1	Aula 101
2	1	Laboratorio
3	2	Campus Norte

Tabla: dispositivos

dispositivo_id (PK)	organization_id (FK)	ubicacion_id (FK, opcional)	nombre	numero_serie (UNIQUE)
101	1	1	Sensor Aula 1	SN-A001
102	1	2	Sensor Aula 2	SN-A002
201	2	3	Sensor Campus	SN-B001
301	3	NULL	Gateway	SN-C001



Las referencias del modelo se convierten en claves foráneas.

Tabla de crecimiento constante

Registrar telemetría

```
CREATE TABLE mediciones (  
  medicion_id BIGINT  
    GENERATED ALWAYS AS IDENTITY  
    PRIMARY KEY,  
  dispositivo_id BIGINT NOT NULL,  
  variable TEXT NOT NULL,  
  valor NUMERIC NOT NULL,  
  unidad TEXT,  
  timestamp TIMESTAMPTZ NOT NULL,  
  
  FOREIGN KEY (dispositivo_id)  
    REFERENCES dispositivos(dispositivo_id)  
);
```

Tabla: dispositivos

dispositivo_id PK	nombre	numero_serie UNIQUE
101	Sensor Aula 1	SN-A001
102	Sensor Aula 2	SN-A002
201	Sensor Campus	SN-B001

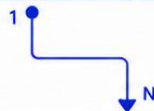


Tabla: mediciones

medicoo_id PK	dispositivo_id FK	variable	valor	unidad	timestamp TIMESTAMPTZ
1	101	temperatura	23.4	°C	2026-09-03 10:00:00+00
2	101	humedad	45.2	%	2026-09-03 10:00:10+00
3	102	temperatura	22.1	°C	2026-09-03 10:00:15+00
4	201	presion	1013.2	hPa	2026-09-03 10:00:20+00
5	101	temperatura	23.6	°C	2026-09-03 10:00:20+00
...	...	...	...	...	...



Cada medición queda asociada
al dispositivo que la generó.

Dependencias entre tablas

No siempre podemos crear todo en cualquier orden

Si dispositivos referencia a organizaciones:



La tabla referenciada debe existir antes de declarar una clave foránea hacia ella.

```
CREATE TABLE organizaciones (
  organizacion_id BIGINT
    GENERATED ALWAYS AS IDENTITY
    PRIMARY KEY,
  nombre TEXT NOT NULL
);
```

```
CREATE TABLE dispositivos (
  dispositivo_id BIGINT
    GENERATED ALWAYS AS IDENTITY
    PRIMARY KEY,
  organizacion_id BIGINT NOT NULL,
  nombre TEXT NOT NULL,
  numero_serie TEXT NOT NULL UNIQUE,

  FOREIGN KEY (organizacion_id)
    REFERENCES organizaciones(organizacion_id)
);
```

```
CREATE TABLE mediciones (
  medicion_id BIGINT
    GENERATED ALWAYS AS IDENTITY
    PRIMARY KEY,
  dispositivo_id BIGINT NOT NULL,
  variable TEXT NOT NULL,
  valor NUMERIC NOT NULL,
  unidad TEXT,
  timestamp TIMESTAMPTZ NOT NULL,

  FOREIGN KEY (dispositivo_id)
    REFERENCES dispositivos(dispositivo_id)
);
```

Tabla: organizaciones

organizacion_id PK	nombre
1	Escuela Técnica
2	Universidad
3	Empresa Solar



Tabla: dispositivos

dispositivo_id PK	organizacion_id FK	nombre
101	1	Sensor Aula 1
102	1	Sensor Aula 2
201	2	Sensor Campus



Tabla: mediciones

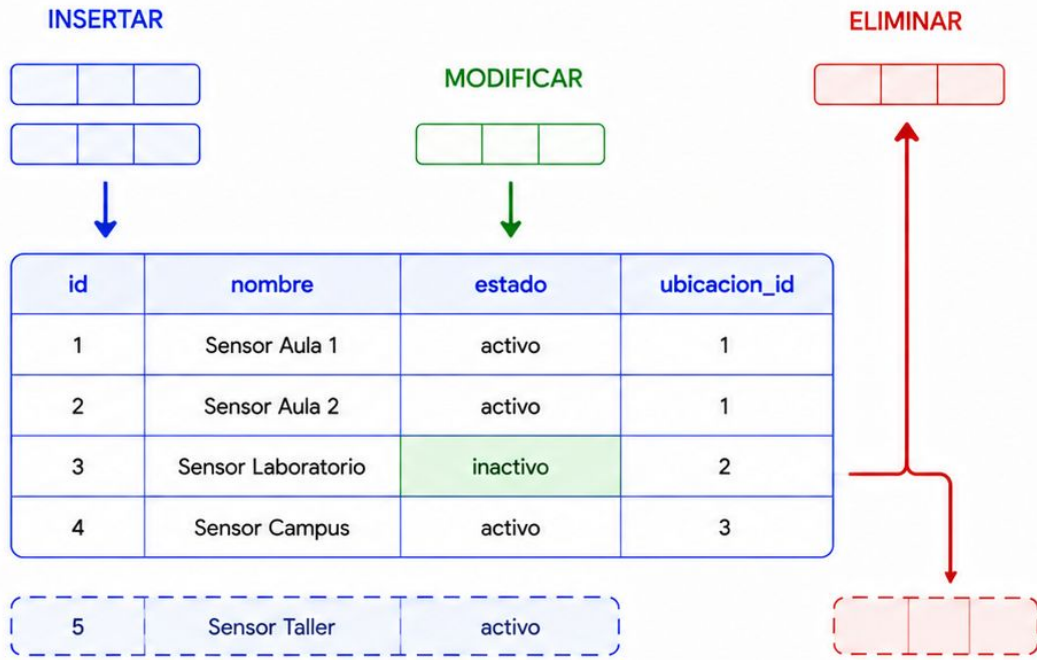
medicion_id PK	dispositivo_id FK	variable	valor	timestamp
1	101	temperatura	23.4	2026-09-03 10:00:00+00
2	101	humedad	45.2	2026-09-03 10:00:10+00
3	102	temperatura	22.1	2026-09-03 10:00:15+00
...	...	...	...	...



Las dependencias del modelo también aparecen durante la implementación.

4

Insertar, Modificar y Eliminar



Insertar datos

Crear nuevas filas

```
INSERT INTO organizaciones (nombre)
VALUES ('Facultad de Ingeniería');
```



Tabla: **organizaciones**

id	nombre
1	Dirección General
2	Departamento Académico
3	Facultad de Ingeniería

```
INSERT INTO dispositivos
(organizacion_id, nombre, numero_serie)
VALUES
(1, 'Ambiente 01', 'AMB-001');
```



Tabla: **dispositivos**

dispositivo_id	organizacion_id	nombre	numero_serie
1	1	Proyector Lobby	PRY-100
2	1	Ambiente 01	AMB-001



INSERT agrega nuevas instancias a una estructura existente.

Múltiples filas

Una operación puede incorporar varios registros

```
INSERT INTO mediciones
(dispositivo_id, variable, valor, unidad, timestamp)
VALUES
(1, 'temperatura', 24.6, '°C', NOW()),
(1, 'humedad', 58, '%', NOW()),
(1, 'presion', 1008, 'hPa', NOW());
```



Tabla: mediciones

medicion_id	dispositivo_id	variable	valor	unidad	timestamp
100	2	temperatura	22.1	°C	2025-05-18 09:15:30
101	1	temperatura	24.6	°C	NOW()
102	1	humedad	58	%	NOW()
103	1	presion	1008	hPa	NOW()



La estructura permanece.

Lo que aumenta es la cantidad de datos almacenados.

Modificar datos

Actualizar filas existentes

```
UPDATE dispositivos
SET nombre = 'Aula 204'
WHERE dispositivo_id = 1;
```

La condición determina qué filas serán modificadas.

Tabla: **dispositivos**

Antes de la actualización

dispositivo_id	organizacion_id	nombre	numero_serie
1	10	Ambiente 01	SN-1001
2	10	Laboratorio 02	SN-1002
3	20	Proyector 01	SN-1003



Después de la actualización

dispositivo_id	organizacion_id	nombre	numero_serie
1	10	Aula 204	SN-1001
2	10	Laboratorio 02	SN-1002
3	20	Proyector 01	SN-1003



UPDATE cambia información existente sin crear una nueva fila.

Eliminar datos

Quitar filas existentes

```
DELETE FROM mediciones  
WHERE medicion_id = 15;
```

Sin una condición:

```
DELETE FROM mediciones;
```

se intentarían eliminar todas las filas.

Antes

medicion_id	dispositivo_id	variable	valor	unidad	timestamp
...	...	...	...	...	...
13	1	temperatura	24.6	°C	2026-09-03 10:00:00+00
15	1	humedad	58	%	2026-09-03 10:00:10+00
16	1	presion	1008	hPa	2026-09-03 10:00:20+00
...	...	...	...	...	...



```
DELETE FROM mediciones  
WHERE medicion_id = 15;
```

Después

medicion_id	dispositivo_id	variable	valor	unidad	timestamp
13	1	temperatura	24.6	°C	2026-09-03 10:00:00+00
16	1	presion	1008	hPa	2026-09-03 10:00:20+00
...	...	...	...	...	...



Modificar o eliminar requiere saber exactamente
qué filas serán afectadas.

Una práctica simple

Consultar primero, modificar después

Antes de ejecutar:

```
DELETE FROM mediciones
WHERE dispositivo_id = 3;
```

podemos comprobar:

```
SELECT *
FROM mediciones
WHERE dispositivo_id = 3;
```

Resultado de la consulta

medicion_id	dispositivo_id	variable	valor	unidad	timestamp
101	3	temperatura	23.8	°C	2026-09-03 10:00:00+00
102	3	humedad	55	%	2026-09-03 10:00:10+00
103	3	presion	1007	hPa	2026-09-03 10:00:20+00
104	3	temperatura	24.1	°C	2026-09-03 10:00:30+00
...	...	...	...	...	...

Si todo es correcto,
recién entonces ejecutar DELETE.



La misma condición puede utilizarse
primero para verificar qué datos serán afectados.

5

SELECT, Filtros y Ordenamiento

id	dispositivo_id	variable	valor	unidad	timestamp
1	1	temperatura	23.4	°C	2026-09-03 10:00:00+00
2	1	humedad	45.2	%	2026-09-03 10:00:10+00
3	1	presion	1008	hPa	2026-09-03 10:00:20+00
4	2	temperatura	22.1	°C	2026-09-03 10:00:00+00
5	2	humedad	50	%	2026-09-03 10:00:10+00
6	3	temperatura	24.6	°C	2026-09-03 10:00:00+00
...	...	...	...	...	...



id	variable	valor	unidad
1	temperatura	23.4	°C
2	humedad	45.2	%
3	presion	1008	hPa
...	...	...	...



Nuestra primera consulta

Recuperar información

```
SELECT *  
FROM dispositivos;
```



dispositivo_id	organizacion_id	nombre	numero_serie	estado	ubicacion_id
1	1	Ambiente 01	AMB-001	activo	1
2	1	Ambiente 02	AMB-002	activo	1
3	2	Laboratorio 01	LAB-001	activo	2
4	2	Exterior Norte	EXT-010	inactivo	3
...	...	...	...	...	...

SELECT indica qué queremos obtener.

FROM indica de dónde provienen los datos.



Consultar no modifica los datos almacenados.

No siempre necesitamos todo

Seleccionar la información relevante

```
SELECT nombre, numero_serie  
FROM dispositivos;
```



Resultado:

nombre	numero_serie
Aula 204	AMB-001
Laboratorio	AMB-002



Una consulta debería recuperar
solamente la información que necesita.

Nombres para el resultado

Hacer la salida más clara

```
SELECT
```

```
  nombre AS dispositivo,
```

```
 numeroserie AS serie
```

```
FROM dispositivos;
```



dispositivo	serie
Aula 204	AMB-001
Laboratorio 01	AMB-002

Los alias modifican cómo se presenta el resultado.

No modifican la estructura de la tabla.



Podemos adaptar el resultado
sin cambiar los datos originales.

Filtrar filas

Elegir qué registros cumplen una condición

```
SELECT *  
FROM mediciones  
WHERE variable = 'temperatura';
```



Resultado: mediciones de temperatura

medicion_id	dispositivo_id	variable	valor	unidad	timestamp
1	1	temperatura	23.4	°C	2026-09-03 10:00:00+00
4	2	temperatura	26.1	°C	2026-09-03 10:00:00+00
7	3	temperatura	25.8	°C	2026-09-03 10:00:00+00
10	1	temperatura	24.6	°C	2026-09-03 10:00:00+00
...	...	...	...	...	...

También podemos comparar valores:

```
SELECT *  
FROM mediciones  
WHERE valor > 25
```



WHERE reduce el conjunto de filas según una condición.

Filtros más específicos

AND y OR

```
SELECT *  
FROM mediciones  
WHERE variable = 'temperatura'  
AND valor > 25;
```



medicion_id	dispositivo_id	variable	valor	unidad	timestamp
4	2	temperatura	26.1	°C	2026-09-03 10:00:00+00
7	3	temperatura	25.8	°C	2026-09-03 10:00:00+00
12	1	temperatura	27.3	°C	2026-09-03 10:00:00+00
...	...	...	...	...	...

Otro ejemplo:

```
SELECT *  
FROM mediciones  
WHERE variable = 'temperatura'  
OR variable = 'humedad'
```



medicion_id	dispositivo_id	variable	valor	unidad	timestamp
1	1	temperatura	23.4	°C	2026-09-03 10:00:00+00
2	1	humedad	45.2	%	2026-09-03 10:00:10+00
4	2	temperatura	26.1	°C	2026-09-03 10:00:00+00
...	...	...	...	...	...



Las condiciones pueden combinarse para representar preguntas más precisas.

Expresar condiciones

IN, BETWEEN y comparaciones

`WHERE variable IN ('temperatura', 'humedad')`

`WHERE valor BETWEEN 20 AND 30`

`WHERE dispositivo_id <> 5`



medicion_id	dispositivo_id	variable	valor	unidad	timestamp
1	1	temperatura	23.4	°C	2026-09-03 10:00:00+00
2	1	humedad	45.2	%	2026-09-03 10:00:10+00
3	2	presion	1013.5	hPa	2026-09-03 10:00:15+00
4	2	temperatura	26.1	°C	2026-09-03 10:00:00+00
5	3	humedad	38.7	%	2026-09-03 10:00:05+00
6	4	temperatura	21.9	°C	2026-09-03 10:00:00+00
...	...	...	...	...	...



SQL ofrece diferentes formas de expresar el conjunto de datos que buscamos.

Cuando no existe un valor

NULL no se compara como un dato convencional

Incorrecto:

```
WHERE ubicacion_id = NULL
```



Correcto:

```
WHERE ubicacion_id IS NULL
```

dispositivo_id	nombre	ubicacion_id
1	Aula 204	2
2	Laboratorio	1
3	Pasillo	NULL
4	Patio	NULL
...	...	...



NULL representa ausencia de valor
y requiere operadores específicos.

Ordenar resultados

Controlar cómo se presentan las filas

```
SELECT *  
FROM mediciones  
ORDER BY timestamp DESC;
```

También:

```
ORDER BY valor ASC;
```

medicion_id	dispositivo_id	variable	valor	unidad	timestamp
105	3	temperatura	23.8	°C	2024-05-07 14:30:15
104	1	humedad	56.2	%	2024-05-07 14:15:00
103	2	presion	1012.4	hPa	2024-05-07 14:00:45
102	1	temperatura	22.6	°C	2024-05-07 13:45:30
101	3	humedad	52.1	%	2024-05-07 13:30:10

DESC
(más reciente
primero)



ASC
(menor a mayor)



El orden del resultado debe pedirse explícitamente cuando resulta importante.

Filtrar por tiempo

Una necesidad frecuente en IoT

```
SELECT *  
FROM mediciones  
WHERE dispositivo_id = 1  
      AND timestamp >= '2026-09-01 00:00:00'  
      AND timestamp <  '2026-09-02 00:00:00'  
ORDER BY timestamp;
```



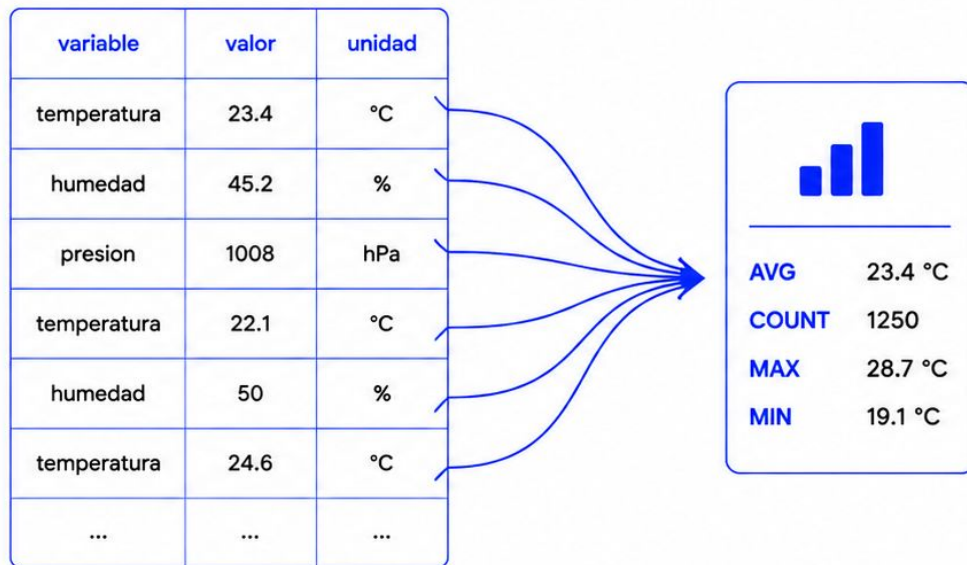
medicion_id	dispositivo_id	variable	valor	timestamp	unidad
1201	1	temperatura	22.3	2026-09-01 00:15:00	°C
1202	1	humedad	48.1	2026-09-01 00:15:00	%
1210	1	temperatura	23.1	2026-09-01 06:00:00	°C
1211	1	humedad	50.2	2026-09-01 06:00:00	%
1235	1	temperatura	24.7	2026-09-01 18:30:00	°C
...	...	...	...	...	...



El timestamp permite consultar
qué ocurrió dentro de un período determinado.

6

Agregaciones y Agrupamientos



Resumir muchas filas

Obtener métricas a partir de los datos

FUNCIONES DE AGREGACIÓN

COUNT()

AVG()

MIN()

MAX()

SUM()

Ejemplo:

```
SELECT AVG(valor)
FROM mediciones
WHERE variable = 'temperatura';
```

Tabla: mediciones

timestamp	variable	valor
2025-05-20 10:00	temperatura	22.1 °C
2025-05-20 10:05	temperatura	23.4 °C
2025-05-20 10:10	temperatura	21.8 °C
2025-05-20 10:15	temperatura	23.0 °C
2025-05-20 10:20	temperatura	22.6 °C
...	...	...



Una agregación transforma varias filas en información resumida.

Agrupar antes de calcular

Obtener una métrica por cada grupo

```
SELECT
    dispositivo_id,
    AVG(valor)
FROM mediciones
WHERE variable = 'temperatura'
GROUP BY dispositivo_id;
```



dispositivo_id	avg
1	23.71
2	24.85
3	25.18
4	22.60
...	...

Ahora obtenemos un promedio por dispositivo.



GROUP BY define sobre qué conjuntos de filas se calcula cada agregación.

Agrupar por varios criterios

Separar los resultados

```
SELECT
    dispositivo_id,
    variable,
    AVG(valor)
FROM mediciones
GROUP BY dispositivo_id, variable;
```



dispositivo_id	variable	avg
1	temperatura	23.71
1	humedad	48.15
2	temperatura	24.85
2	humedad	52.30
3	temperatura	25.18
3	humedad	49.90
...	...	...

Podemos obtener una métrica diferente para cada combinación.



La agrupación determina el nivel de detalle del resultado.

Filtrar grupos

Aplicar condiciones después de agregar

```
SELECT
    dispositivo_id,
    AVG(valor) AS promedio
FROM mediciones
WHERE variable = 'temperatura'
GROUP BY dispositivo_id
HAVING AVG(valor) > 25;
```



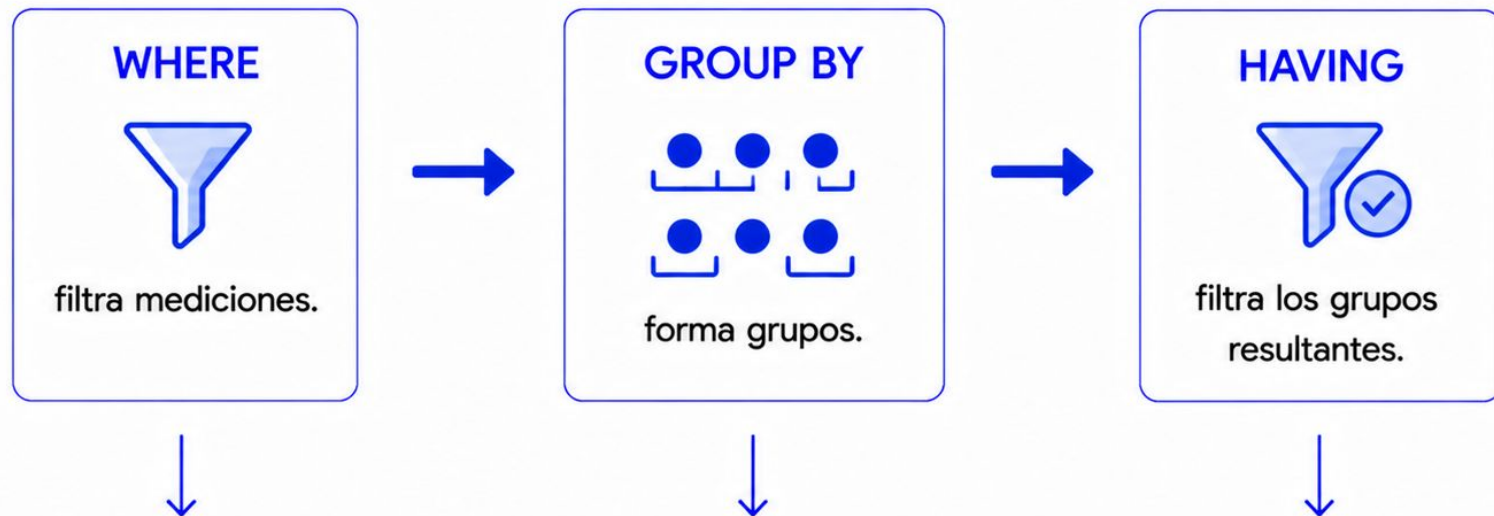
dispositivo_id	promedio
1	25.42
3	25.87
4	25.11
...	...



HAVING permite conservar solamente los grupos que cumplen una condición.

Dos momentos diferentes

Filas antes, grupos después



WHERE actúa antes de agrupar.

HAVING actúa sobre el resultado agregado.

Preguntas resueltas

Utilizar los datos almacenados

Podemos consultar, por ejemplo:

- cantidad de mediciones por dispositivo;
- temperatura promedio;
- valor máximo registrado;
- valor mínimo registrado;
- promedio por variable;
- dispositivos cuyo promedio supera un umbral.

Tabla de mediciones

id	dispositivo_id	variable	valor	timestamp
1	1	temperatura	22.3	2026-09-01 00:15:00
2	1	humedad	48.1	2026-09-01 00:15:00
3	1	temperatura	23.1	2026-09-01 01:00:00
4	2	temperatura	24.0	2026-09-01 00:10:00
5	2	temperatura	26.8	2026-09-01 01:20:00
...	...	...	...	...



COUNT()
AVG()
MIN()
MAX()
GROUP BY
HAVING

Métricas obtenidas



Promedio temperatura
23.71 °C



Valor máximo
28.4 °C



Valor mínimo
18.2 °C

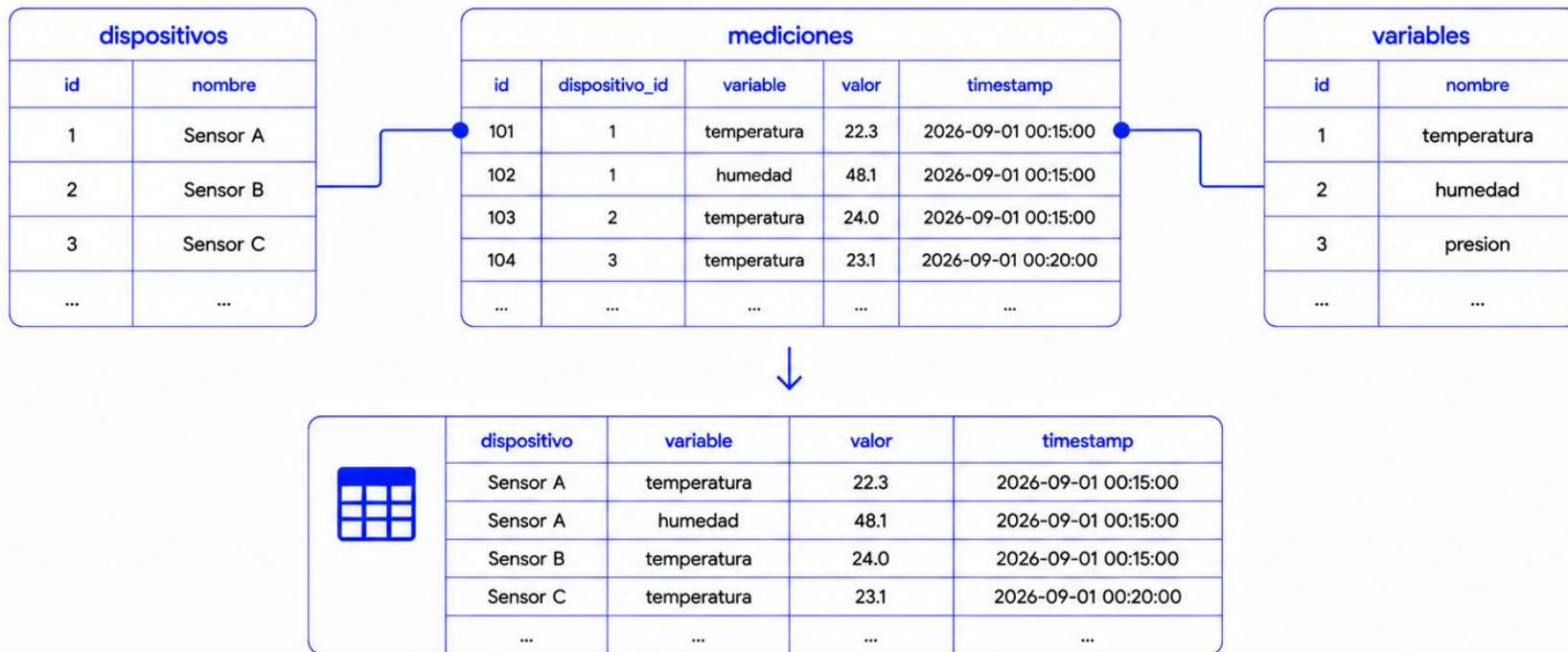


Cantidad de mediciones
15.842



SQL permite transformar filas almacenadas en indicadores útiles para la aplicación.

7 JOIN



Consultas complejas

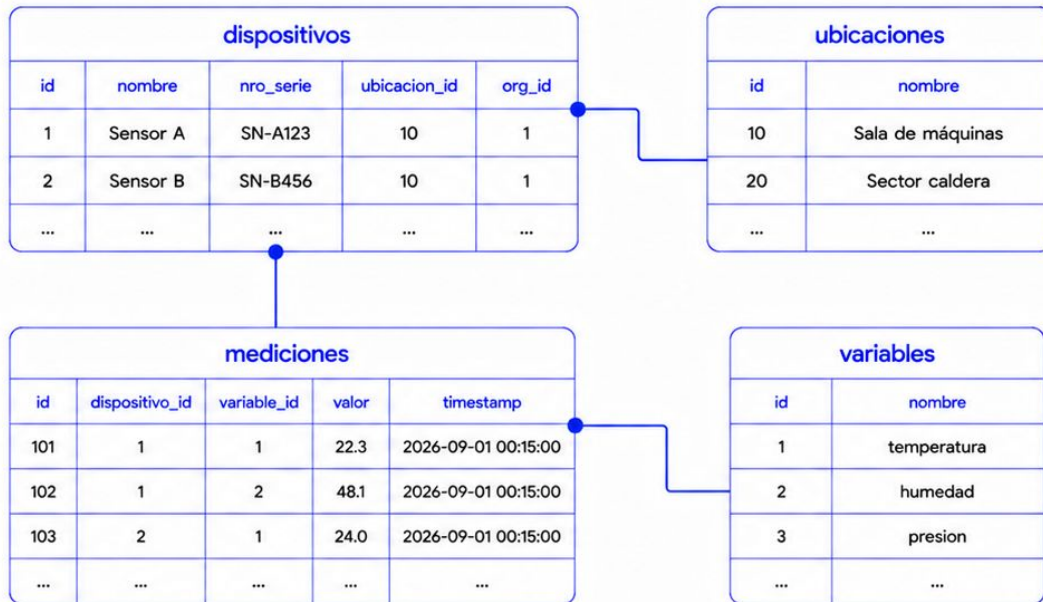
Recuperar el contexto completo

Una medición contiene: `dispositivo_id`

pero quizás necesitamos mostrar:

- nombre del dispositivo;
- número de serie;
- ubicación;
- organización;
- variable;
- valor;
- timestamp.

Esa información no está almacenada en una única tabla.



Las relaciones permiten reconstruir información distribuida entre varias tablas.

Combinar tablas relacionadas

Utilizar las claves para conectar filas

```
SELECT
```

```
  d.nombre,  
  m.variable,  
  m.valor,  
  m.timestamp
```

```
FROM mediciones m
```

```
JOIN dispositivos d
```

```
  ON m.dispositivo_id = d.dispositivo_id;
```

mediciones

id	dispositivo_id	variable	valor	timestamp
101	1	temperatura	22.3	2026-09-01 00:15:00
102	1	humedad	48.1	2026-09-01 00:15:00
103	2	temperatura	24.0	2026-09-01 00:15:00
104	3	temperatura	23.1	2026-09-01 00:20:00
...	...	...	...	...

dispositivos

dispositivo_id	nombre
1	Sensor A
2	Sensor B
3	Sensor C
...	...



resultado (INNER JOIN)

nombre	variable	valor	timestamp
Sensor A	temperatura	22.3	2026-09-01 00:15:00
Sensor A	humedad	48.1	2026-09-01 00:15:00
Sensor B	temperatura	24.0	2026-09-01 00:15:00
Sensor C	temperatura	23.1	2026-09-01 00:20:00
...	...	...	...



JOIN utiliza una condición para determinar qué filas corresponden entre sí.

Seguir las relaciones

Una consulta puede recorrer el modelo

SELECT

```
d.nombre AS dispositivo,  
u.nombre AS ubicacion,  
m.variable,  
m.valor
```

FROM mediciones m

JOIN dispositivos d

ON m.dispositivo_id = d.dispositivo_id

JOIN ubicaciones u

ON d.ubicacion_id = u.ubicacion_id;

mediciones

id	dispositivo_id	variable	valor
101	1	temperatura	22.3
102	1	humedad	48.1
103	2	temperatura	24.0
104	3	temperatura	23.1
...	...	...	...

dispositivos

dispositivo_id	nombre	ubicacion_id
1	Sensor A	10
2	Sensor B	20
3	Sensor C	10
...	...	...

ubicaciones

ubicacion_id	nombre
10	Sala de máquinas
20	Sector caldera
30	Tanque elevado
...	...



resultado de la consulta

dispositivo	ubicacion	variable	valor
Sensor A	Sala de máquinas	temperatura	22.3
Sensor A	Sala de máquinas	humedad	48.1
Sensor B	Sector caldera	temperatura	24.0
Sensor C	Sala de máquinas	temperatura	23.1
...	...	...	...



Las claves foráneas también indican el camino que podemos recorrer al consultar.

Reconstruir el contexto

De la medición a la organización



mediciones

id	dispositivo_id	variable	valor	timestamp
101	1	temperatura	22.3	2026-09-01 00:15:00
102	1	humedad	48.1	2026-09-01 00:15:00
103	2	temperatura	24.0	2026-09-01 00:15:00
104	3	temperatura	23.1	2026-09-01 00:20:00
...	...	...	...	...

dispositivos

dispositivo_id	nombre	organizacion_id
1	Sensor A	1
2	Sensor B	2
3	Sensor C	1
...	...	...

organizaciones

organizacion_id	nombre
1	Cooperativa Norte
2	Industria Sur
3	Municipalidad Centro
...	...

Consulta:

```
SELECT
    o.nombre AS organizacion,
    d.nombre AS dispositivo,
    m.variable,
    m.valor
FROM mediciones m
JOIN dispositivos d
    ON m.dispositivo_id = d.dispositivo_id
JOIN organizaciones o
    ON d.organizacion_id = o.organizacion_id;
```



organizacion	dispositivo	variable	valor
Cooperativa Norte	Sensor A	temperatura	22.3
Cooperativa Norte	Sensor A	humedad	48.1
Industria Sur	Sensor B	temperatura	24.0
Cooperativa Norte	Sensor C	temperatura	23.1
...	...	...	...



El modelo relacional separa información sin impedir que podamos volver a combinarla.

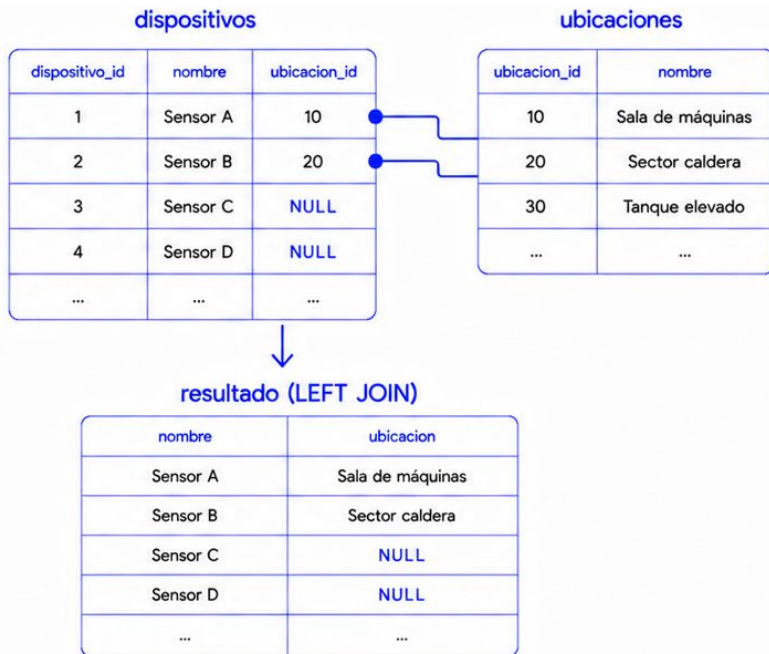
¿Y si la relación es opcional?

Conservar filas aunque falte información relacionada

Un dispositivo puede no tener todavía una ubicación asignada.

```
SELECT
    d.nombre,
    u.nombre AS ubicacion
FROM dispositivos d
LEFT JOIN ubicaciones u
    ON d.ubicacion_id = u.ubicacion_id;
```

El dispositivo seguirá apareciendo aunque
`ubicacion_id` sea `NULL`.



LEFT JOIN permite conservar las filas del lado principal aunque no exista una coincidencia.

Tipos de JOIN

Qué filas queremos conservar al combinar dos tablas

INNER JOIN

Solo filas con coincidencia en A y B.

LEFT JOIN

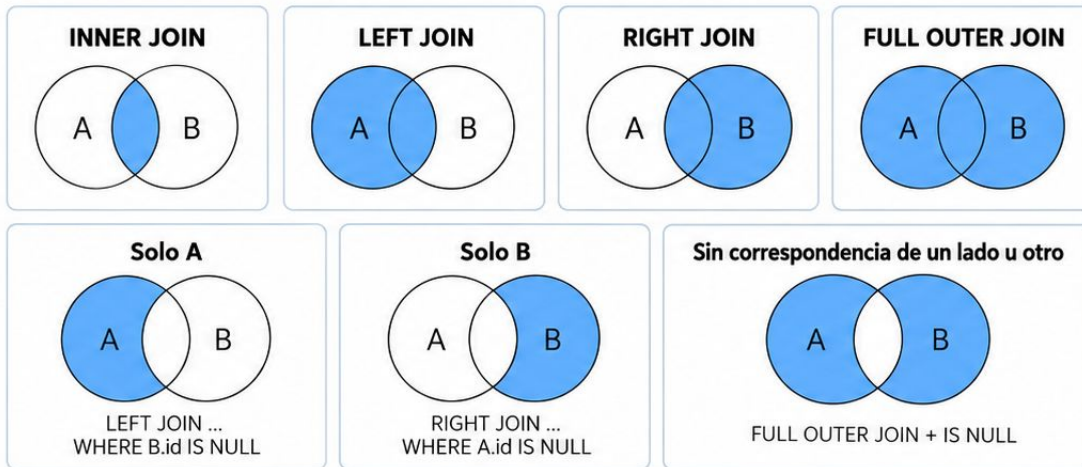
Todas las filas de A y las coincidencias de B.
Si no existe coincidencia, los datos de B aparecen como NULL.

RIGHT JOIN

Todas las filas de B y las coincidencias de A.
Es conceptualmente el inverso de LEFT JOIN.

FULL OUTER JOIN

Conserva todas las filas de A y B, tengan o no coincidencia.



También podemos buscar únicamente las filas sin correspondencia:

- LEFT JOIN ... WHERE B.id IS NULL → filas de A sin coincidencia en B.
- RIGHT JOIN ... WHERE A.id IS NULL → filas de B sin coincidencia en A.
- FULL OUTER JOIN + IS NULL → filas que existen solamente de un lado.



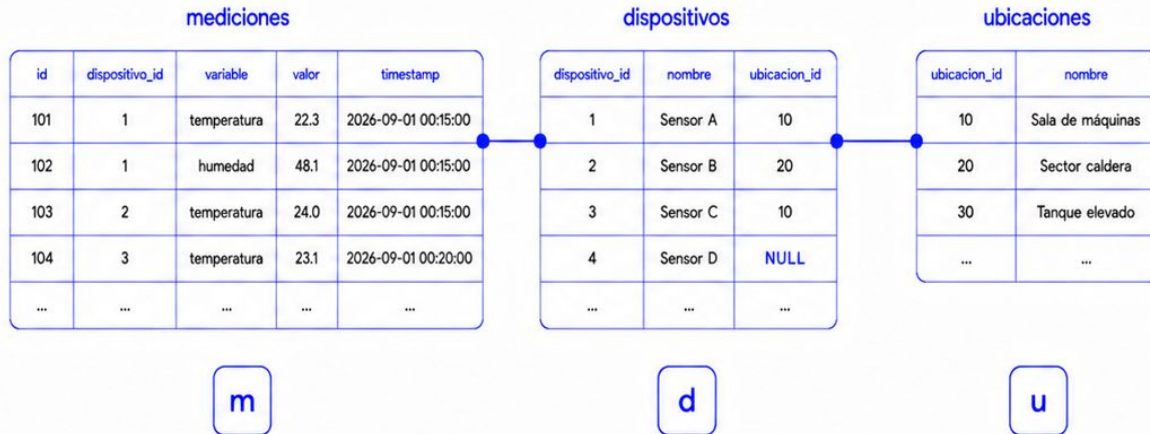
El tipo de JOIN determina qué ocurre con las filas que no encuentran una coincidencia.

Consultas más legibles

Reducir nombres repetidos

```
FROM mediciones m
JOIN dispositivos d
    ON m.dispositivo_id = d.dispositivo_id
JOIN ubicaciones u
    ON d.ubicacion_id = u.ubicacion_id
```

m, d y u son alias.

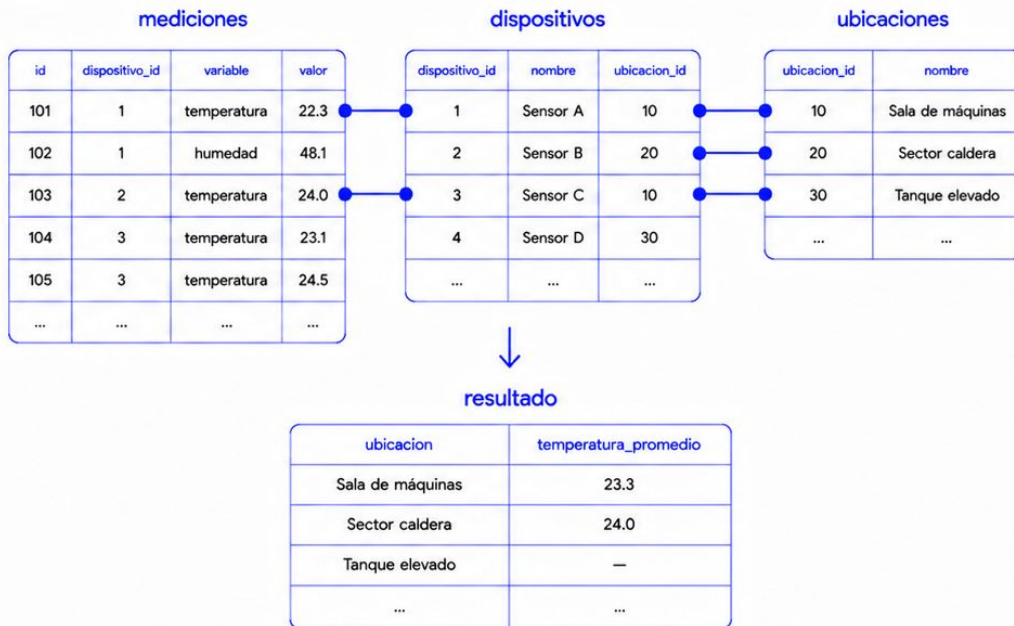


Los alias ayudan a mantener legibles las consultas que utilizan varias tablas.

Responder preguntas del dominio

Relacionar y resumir

```
SELECT
    u.nombre AS ubicacion,
    AVG(m.valor) AS temperatura_promedio
FROM mediciones m
JOIN dispositivos d
    ON m.dispositivo_id = d.dispositivo_id
JOIN ubicaciones u
    ON d.ubicacion_id = u.ubicacion_id
WHERE m.variable = 'temperatura'
GROUP BY u.nombre;
```



Las operaciones SQL pueden combinarse para responder preguntas cada vez más completas.

8

Subconsultas y CTE



SUBCONSULTA / CTE

```
WITH prom_por_disp AS (  
  SELECT  
    m.id_dispositivo,  
    m.id_variable,  
    AVG(m.valor) AS promedio  
  FROM mediciones m  
  GROUP BY m.id_dispositivo, m.id_variable  
)
```

CONSULTA PRINCIPAL

```
SELECT  
  d.id_dispositivo,  
  d.nombre,  
  p.id_variable,  
  p.promedio  
FROM prom_por_disp p  
JOIN dispositivos d  
  ON p.id_dispositivo = d.id_dispositivo  
ORDER BY d.nombre, p.id_variable;
```

RESULTADO

id_dispositivo	nombre	id_variable	promedio
1	Sensor Patio	temperatura	24.3
1	Sensor Patio	humedad	56.8
2	Sensor Invernadero	temperatura	28.7
2	Sensor Invernadero	humedad	72.1
3	Estación Techo	temperatura	22.1
3	Estación Techo	humedad	49.6

Una consulta dentro de otra

Utilizar un resultado como parte de otra operación

```
SELECT *  
FROM mediciones  
WHERE valor >  
    (  
        SELECT AVG(valor)  
        FROM mediciones  
        WHERE variable = 'temperatura'  
    )  
AND variable = 'temperatura';
```

mediciones

variable	valor
temperatura	21.3
temperatura	23.7
temperatura	18.9
temperatura	26.4
temperatura	24.8
temperatura	27.6
temperatura	25.1
temperatura	19.2

subconsulta

AVG(valor) = 24.2

resultado

variable	valor
temperatura	26.4
temperatura	24.8
temperatura	27.6
temperatura	25.1

Se obtienen las filas de temperatura con valor mayor que 24.2.



Una subconsulta permite utilizar el resultado de una consulta dentro de otra.

Pensar por etapas

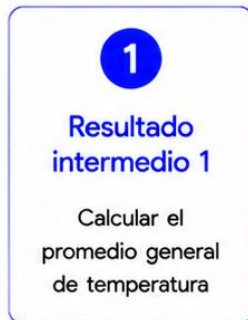
Dividir una pregunta compleja

Pregunta:

¿Qué dispositivos registraron una temperatura superior al promedio general?

Podemos pensar:

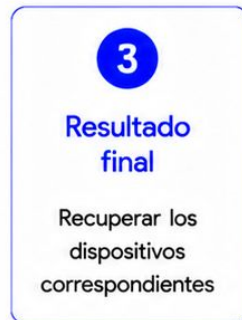
1. calcular el promedio;
2. identificar mediciones superiores;
3. recuperar los dispositivos correspondientes.



promedio_temperatura
23.4



dispositivo_id	valor
3	24.8
7	25.1
...	...



dispositivo_id	nombre
3	Sensor Patio
7	Sensor Techo
...	...



Las consultas complejas suelen resultar más simples cuando podemos dividir el problema.

Common Table Expression

Nombrar un resultado intermedio

```
WITH temperaturas AS (  
    SELECT dispositivo_id, valor  
    FROM mediciones  
    WHERE variable = 'temperatura'  
)  
  
SELECT  
    dispositivo_id,  
    AVG(valor)  
FROM temperaturas  
GROUP BY dispositivo_id;
```

Paso 1: CTE "temperaturas"

dispositivo_id	valor
1	22.3
1	24.8
2	23.1
2	25.0
3	21.9
...	...

nombre: temperaturas

Paso 2: consulta final

dispositivo_id	avg
1	23.55
2	24.05
3	21.90
...	...



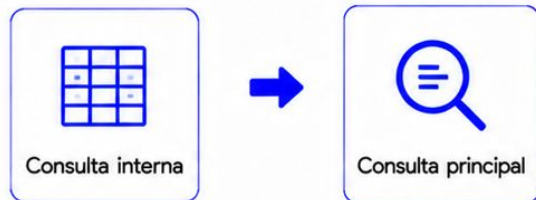
Una CTE permite expresar una consulta compleja mediante pasos con nombre.

Dos formas de construir consultas

Elegir según la claridad

Subconsulta

Útil para resultados intermedios pequeños y directos.



CTE

Puede facilitar la lectura cuando la consulta tiene varias etapas.



La mejor consulta no es necesariamente la más corta.
También debe poder comprenderse y mantenerse.

9

Funciones de Ventana

Una introducción.

Datos originales

dispositivo_id	timestamp	valor
1	2026-09-01 00:00	22.1
1	2026-09-01 00:10	22.8
1	2026-09-01 00:20	23.6
1	2026-09-01 00:30	22.9
1	2026-09-01 00:40	23.2
...	...	...

Ventana
últimas 3 mediciones

2026-09-01 00:00	22.1
2026-09-01 00:10	22.8
2026-09-01 00:20	23.6

Para cada fila, la función de ventana considera un conjunto de filas relacionadas.

Resultado con función de ventana

dispositivo_id	timestamp	valor	promedio_3
1	2026-09-01 00:00	22.1	22.10
1	2026-09-01 00:10	22.8	22.45
1	2026-09-01 00:20	23.6	22.83
1	2026-09-01 00:30	22.9	23.10
1	2026-09-01 00:40	23.2	22.90
...	...	...	...

← Promedio solo con esta fila

← Promedio con 2 filas

← Promedio con 3 filas

← Últimas 3 filas
(00:10, 00:20, 00:30)

← Últimas 3 filas
(00:20, 00:30, 00:40)

Otra forma de calcular

Mantener el detalle original

Con GROUP BY:

muchas filas

dispositivo_id	variable	valor
1	temperatura	22.1
1	temperatura	22.8
1	temperatura	23.6
2	temperatura	24.0
2	temperatura	24.4
...	...	...



un resultado por grupo

dispositivo_id	promedio_temperatura
1	22.83
2	24.20
...	...

Con una función de ventana:

las filas permanecen

dispositivo_id	variable	valor	promedio_temperatura
1	temperatura	22.1	22.83
1	temperatura	22.8	22.83
1	temperatura	23.6	22.83
2	temperatura	24.0	24.20
2	temperatura	24.4	24.20
...	...	...	...



se incorpora un cálculo relacionado.



Una función de ventana calcula sobre un conjunto de filas sin necesariamente reducirlas a una sola.

ROW_NUMBER

Numerar mediciones dentro de cada grupo

```
SELECT
    dispositivo_id,
    valor,
    timestamp,
    ROW_NUMBER() OVER (
        PARTITION BY dispositivo_id
        ORDER BY timestamp DESC
    ) AS posicion
FROM mediciones
WHERE variable = 'temperatura';
```



Resultado (ordenado por dispositivo y timestamp descendente)

dispositivo_id	valor	timestamp	posicion
1	23.6	2026-09-01 00:20:00	1
1	22.8	2026-09-01 00:10:00	2
1	22.1	2026-09-01 00:00:00	3
...	...	...	...
2	24.4	2026-09-01 00:30:00	1
2	24.0	2026-09-01 00:20:00	2
2	23.2	2026-09-01 00:10:00	3
...	...	...	...



Podemos analizar cada dispositivo manteniendo visibles sus mediciones individuales.

LAG

Mirar una fila anterior

```
SELECT
    dispositivo_id,
    timestamp,
    valor,
    LAG(valor) OVER (
        PARTITION BY dispositivo_id
        ORDER BY timestamp
    ) AS valor_anterior
FROM mediciones
WHERE variable = 'temperatura';
```



Resultado (ordenado por dispositivo y timestamp)

dispositivo_id	timestamp	valor	valor_anterior
1	2026-09-01 00:00:00	22.1	NULL
1	2026-09-01 00:10:00	22.8	22.1
1	2026-09-01 00:20:00	23.6	22.8
1	2026-09-01 00:30:00	22.9	23.6
...	...	...	...
2	2026-09-01 00:00:00	23.5	NULL
2	2026-09-01 00:10:00	24.4	23.5
2	2026-09-01 00:20:00	24.0	24.4
2	2026-09-01 00:30:00	23.2	24.0
...	...	...	...



Las funciones de ventana permiten analizar relaciones entre filas dentro de una secuencia.

10 Operaciones Controladas

Transacciones y consultas parametrizadas.

BEGIN



```
INSERT INTO mediciones (dispositivo_id, variable, valor, timestamp)
VALUES (1, 'temperatura', 22.8, '2026-09-01 00:10:00');
```



```
UPDATE dispositivos
SET ultima_medicion = '2026-09-01 00:10:00'
WHERE id = 1;
```



```
INSERT INTO eventos (dispositivo_id, tipo, detalle, timestamp)
VALUES (1, 'nueva_medicion', 'Temperatura registrada',
'2026-09-01 00:10:00');
```

COMMIT

Varias operaciones, una unidad

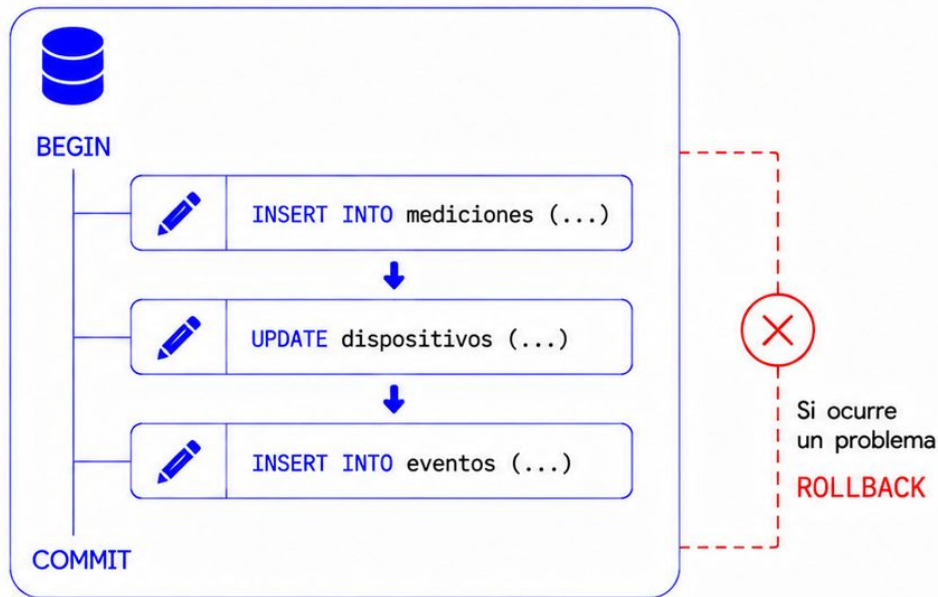
Confirmar todo o deshacer todo

```
BEGIN;  
-- operaciones  
COMMIT;
```

Si ocurre un problema:

```
ROLLBACK;
```

Ejemplo de una transacción



Una transacción permite tratar varias operaciones como una única unidad de trabajo.

Registrar información relacionada

Evitar resultados parciales

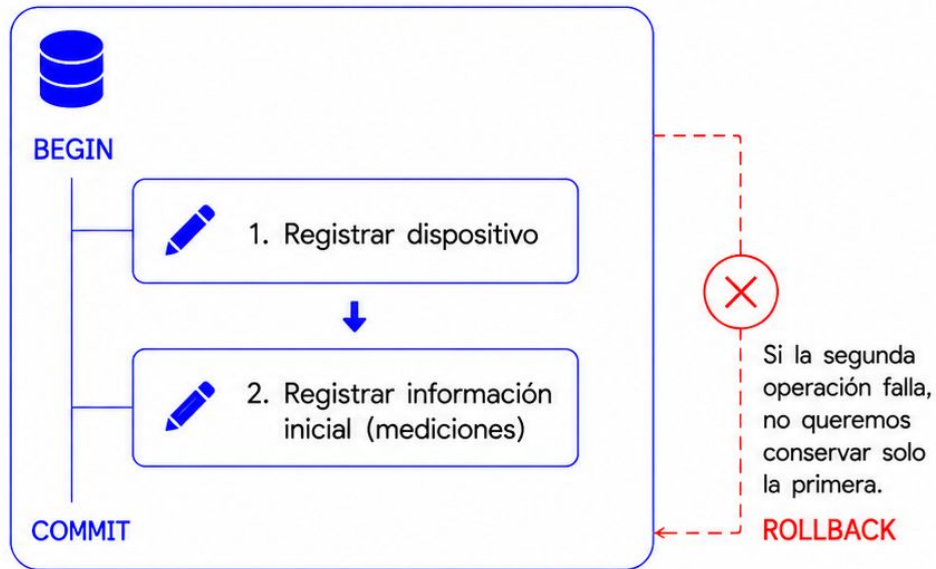
Supongamos que una operación debe:

1. registrar un dispositivo;
2. registrar información inicial asociada.

Si la segunda operación falla, quizás no queremos conservar solamente la primera.

```
BEGIN;  
INSERT INTO dispositivos ...;  
INSERT INTO mediciones ...;  
COMMIT;
```

Operación lógica: registrar dispositivo
e información inicial asociada



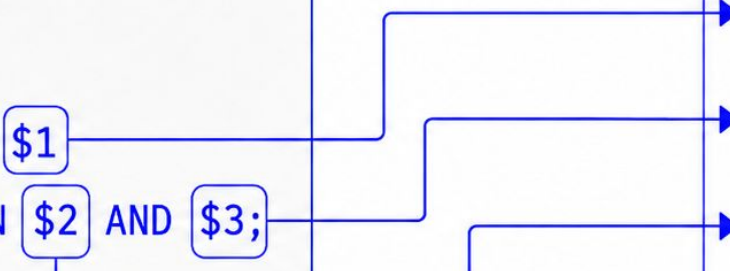
Una operación lógica puede requerir varias instrucciones SQL.

Consultas parametrizadas

Separar la consulta de sus parámetros

Consulta:

```
SELECT *  
FROM mediciones  
WHERE dispositivo_id = $1  
AND timestamp BETWEEN $2 AND $3;
```



Parámetros:

\$1 → 15
\$2 → fecha inicial
\$3 → fecha final

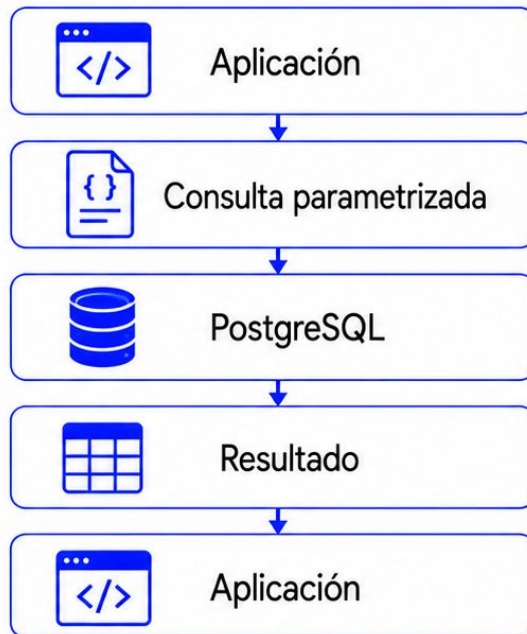


La estructura de la consulta permanece estable mientras cambian los valores utilizados.

Más allá de la consola

Las aplicaciones también ejecutan consultas

Las aplicaciones envían instrucciones y valores al SGBD y reciben resultados.



Las consultas parametrizadas permiten utilizar SQL de manera controlada desde el software.

11 Práctica Guiada

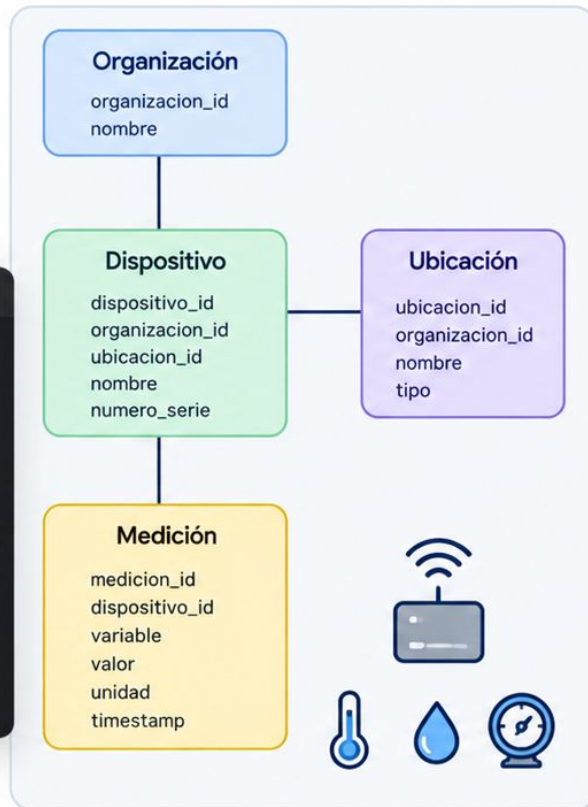
Implementar y consultar el modelo desarrollado anteriormente.



```
psql -d iotdb

iotdb=> SELECT d.nombre,
           m.variable,
           m.valor,
           m.timestamp
FROM mediciones m
JOIN dispositivos d ON d.dispositivo_id = m.dispositivo_id
WHERE d.dispositivo_id = 1
ORDER BY m.timestamp DESC
LIMIT 10;

iotdb=> |
```



Práctica

Del modelo lógico a consultas reales

Vamos a:

- crear las tablas;
- implementar claves y restricciones;
- cargar datos simulados;
- consultar dispositivos y mediciones;
- modificar información;
- combinar tablas;
- construir métricas;
- consultar períodos de tiempo;
- probar una consulta más avanzada.



Al finalizar tendremos una implementación funcional del modelo trabajado en la clase anterior.

Paso 1

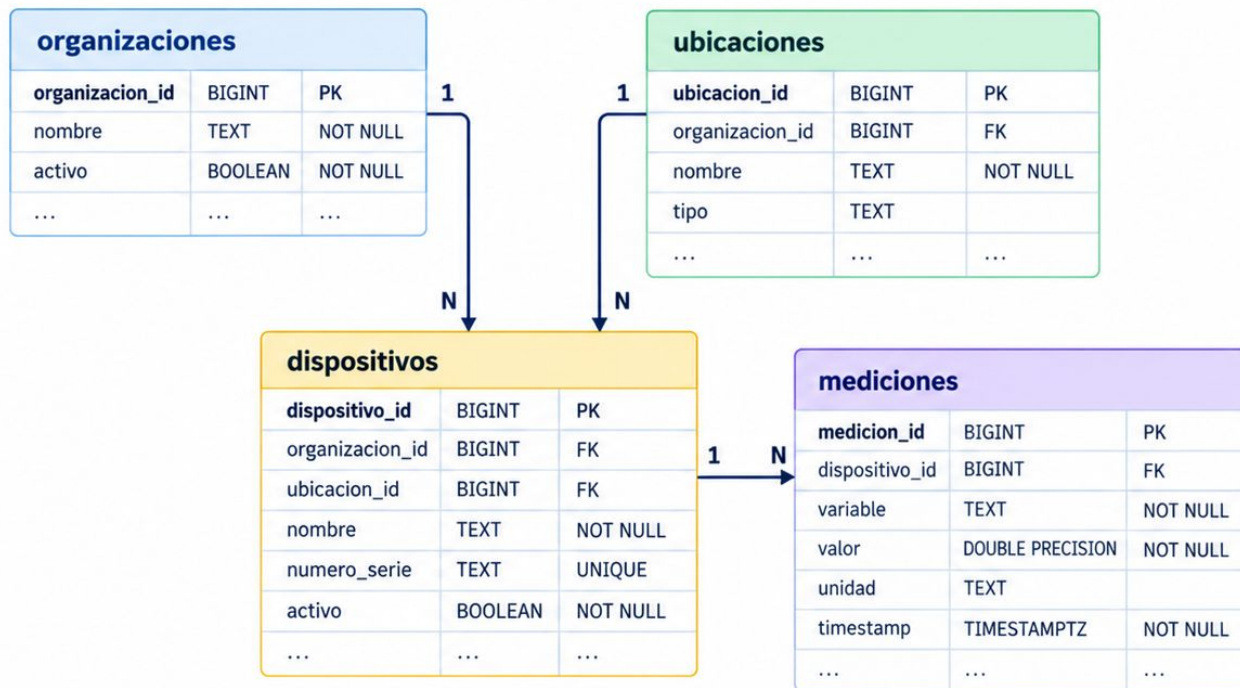
Crear el esquema

Implementaremos:

- organizaciones;
- ubicaciones;
- dispositivos;
- mediciones.

Vamos a verificar:

- claves primarias;
- claves foráneas;
- obligatoriedad;
- unicidad;
- tipos de datos.



El primer objetivo es que la estructura implemente correctamente el modelo.

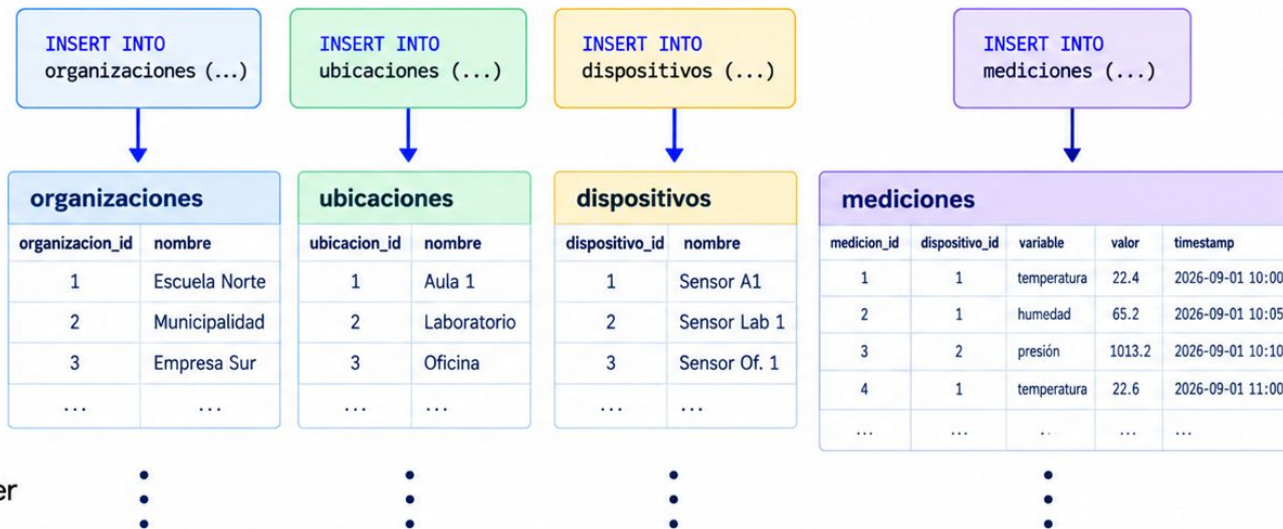
Paso 2

Cargar datos simulados

Vamos a registrar:

- organizaciones;
- ubicaciones;
- varios dispositivos;
- mediciones de temperatura;
- humedad;
- presión.

Las mediciones tendrán timestamps diferentes para poder realizar consultas temporales.



Necesitamos datos suficientes para que las consultas produzcan resultados interesantes.

Paso 3

Consultar y filtrar

Vamos a obtener:

- todos los dispositivos;
- dispositivos de una organización;
- mediciones de una variable;
- mediciones superiores a determinado valor;
- mediciones dentro de un período;
- últimas mediciones ordenadas por timestamp.

1. Todos los dispositivos

```
SELECT *  
FROM dispositivos  
ORDER BY dispositivo_id;
```

dispositivo_id	nombre	organizacion_id	ubicacion_id
1	Sensor A1	1	1
2	Sensor Lab	1	2
3	Sensor Ofic.	2	3
...	...	...	...

2. Dispositivos de una organización

```
SELECT *  
FROM dispositivos  
WHERE organizacion_id = 1  
ORDER BY nombre;
```

dispositivo_id	nombre	organizacion_id	ubicacion_id
1	Sensor A1	1	1
2	Sensor A2	1	2
...	...	...	...

3. Mediciones de una variable

```
SELECT *  
FROM mediciones  
WHERE variable = 'temperatura'  
ORDER BY timestamp DESC;
```

medicion_id	dispositivo_id	variable	valor	timestamp
101	1	temperatura	22.4	2026-09-01 10:00
98	2	temperatura	23.1	2026-09-01 09:45
95	1	temperatura	21.8	2026-09-01 09:30
...	...	...	...	...

4. Mediciones superiores a un valor

```
SELECT *  
FROM mediciones  
WHERE valor > 30  
ORDER BY valor DESC;
```

medicion_id	dispositivo_id	variable	valor	timestamp
87	3	humedad	65.2	2026-09-01 11:20
76	2	temperatura	31.5	2026-09-01 10:10
71	1	presión	1013.2	2026-09-01 09:00
...	...	...	...	...

5. Mediciones dentro de un período

```
SELECT *  
FROM mediciones  
WHERE timestamp BETWEEN '2026-09-01'  
AND '2026-09-02'  
ORDER BY timestamp;
```

medicion_id	dispositivo_id	variable	valor	timestamp
90	1	temperatura	22.1	2026-09-01 08:00
91	1	temperatura	22.4	2026-09-01 10:00
92	2	humedad	60.5	2026-09-01 12:00
...	...	...	...	...

6. Últimas mediciones

```
SELECT *  
FROM mediciones  
ORDER BY timestamp DESC  
LIMIT 10;
```

medicion_id	dispositivo_id	variable	valor	timestamp
120	2	presión	1012.8	2026-09-01 12:30
119	1	humedad	58.1	2026-09-01 12:25
118	3	temperatura	22.7	2026-09-01 12:20
...	...	...	...	...



Primero vamos a aprender a recuperar exactamente las filas que necesitamos.

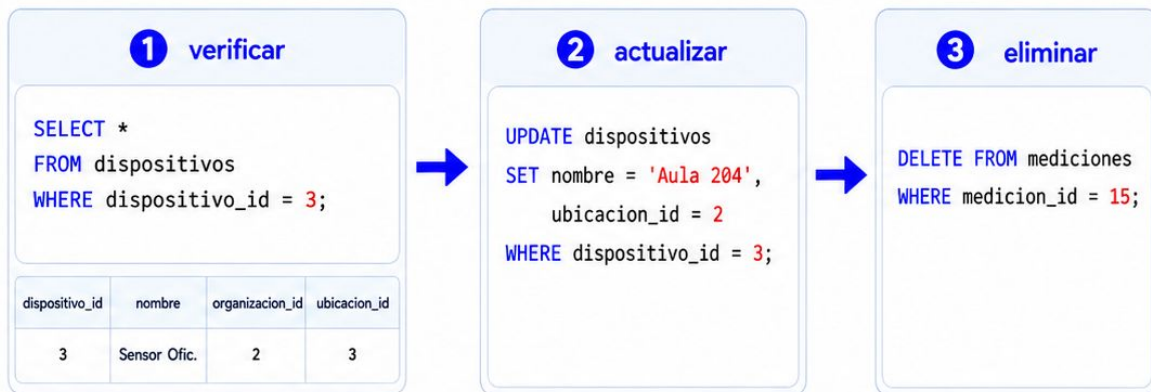
Paso 4

Modificar información

Vamos a:

- modificar el nombre de un dispositivo;
- asignarle una ubicación;
- actualizar información existente;
- eliminar un registro de prueba.

Antes de modificar o eliminar, verificaremos las filas mediante `SELECT`.



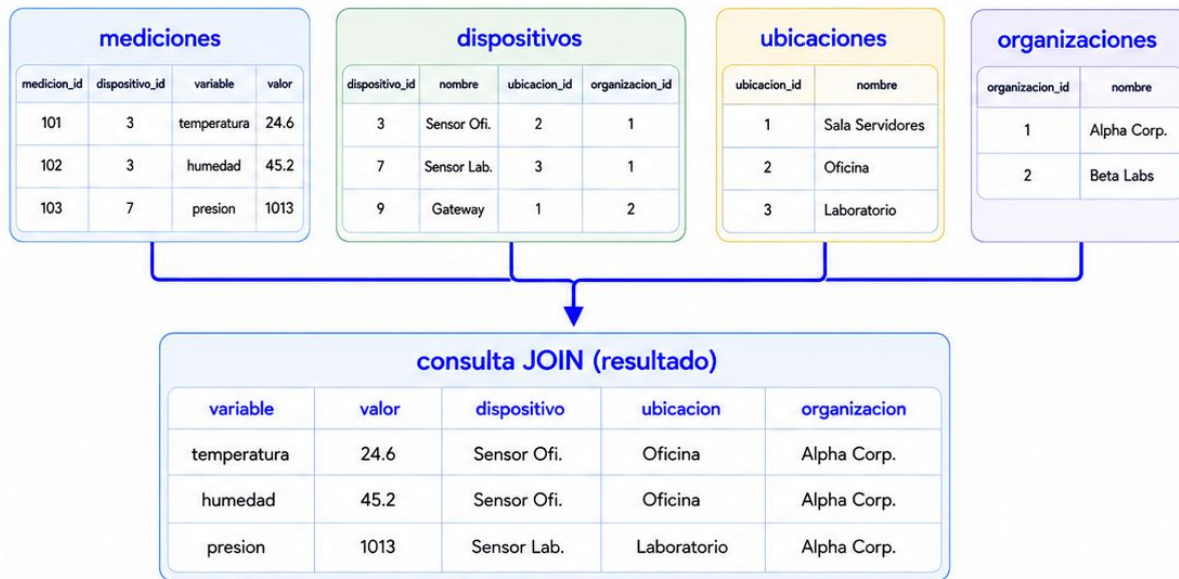
Consultar antes de modificar ayuda a comprobar el alcance de una operación.

Paso 5

Combinar información

Construiremos consultas para mostrar:

- medición + dispositivo;
- dispositivo + ubicación;
- dispositivo + organización;
- medición + dispositivo + ubicación.



JOIN nos permitirá reconstruir el contexto completo a partir de datos relacionados.

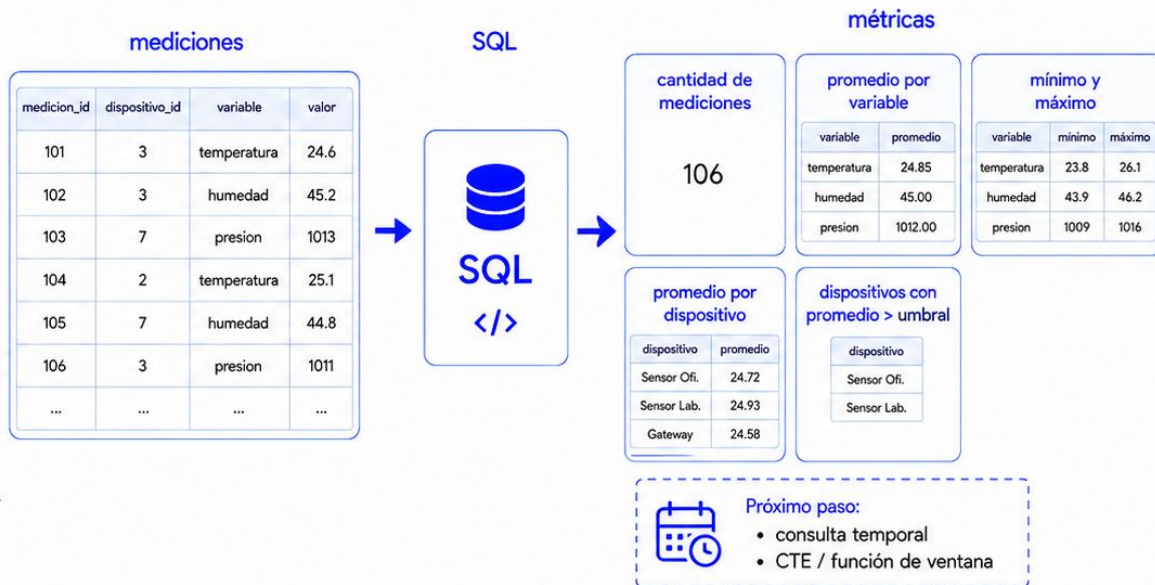
Paso 6

Construir métricas

Vamos a obtener:

- cantidad de mediciones;
- promedio por variable;
- mínimo y máximo;
- promedio por dispositivo;
- dispositivos que superan determinado promedio.

Después realizaremos una consulta temporal y una consulta utilizando CTE o función de ventana.



El objetivo final no es solamente recuperar filas, sino obtener información útil a partir de ellas.



Para ir cerrando..

Errores frecuentes

Problemas típicos al comenzar con SQL

- Insertar una FK que no existe.
- Confundir el identificador interno con otros atributos.
- Utilizar `UPDATE` o `DELETE` sin revisar la condición.
- Comparar `NULL` con `=`.
- Suponer que un resultado viene ordenado.
- Utilizar una agregación sin pensar qué representa cada grupo.
- Hacer un `JOIN` sin una condición correcta.
- Construir consultas demasiado complejas sin dividir el problema.

1. Insertar una FK que no existe

```
INSERT INTO dispositivos  
(organizacion_id, nombre)  
VALUES (999, 'Sensor A1');
```

❗ Error: la organización 999 no existe.

3. UPDATE sin condición

```
UPDATE dispositivos  
SET nombre = 'Sensor X';
```

❗ Se actualizarán todas las filas.

5. Resultado sin orden

```
SELECT *  
FROM mediciones  
LIMIT 10;
```

❗ El resultado no tiene un orden garantizado.

7. JOIN sin condición correcta

```
SELECT *  
FROM dispositivos d, ubicaciones u;
```

❗ Falta la condición de unión (JOIN ... ON ...).

2. Confundir identificadores

```
SELECT *  
FROM dispositivos  
WHERE nombre = 1;
```

❗ El identificador es `dispositivo_id`, no el nombre.

4. Comparar NULL con =

```
SELECT *  
FROM dispositivos  
WHERE ubicacion_id = NULL;
```

❗ La comparación con NULL debe hacerse con `IS NULL`.

6. Agregación sin contexto

```
SELECT variable, AVG(valor)  
FROM mediciones;
```

❗ ¿Qué representa cada promedio? Considerar `GROUP BY`.

8. Consultas demasiado complejas

```
SELECT ...  
FROM ...  
WHERE ... AND ... AND ...
```

❗ Dividir el problema en pasos más simples (CTE o subconsultas).



Una consulta puede ser sintácticamente válida y aun así responder incorrectamente la pregunta.

Buenas prácticas

Trabajar con SQL de manera clara

- Seleccionar solamente las columnas necesarias.
- Utilizar nombres y alias comprensibles.
- Verificar filtros antes de modificar datos.
- Seguir las claves para construir JOIN.
- Pensar qué representa cada agrupación.
- Formatear consultas complejas.
- Dividir problemas grandes en etapas.
- Utilizar parámetros desde las aplicaciones.

Ejemplo de consulta

```
SELECT
  d.nombre      AS dispositivo,
  u.nombre      AS ubicacion,
  o.nombre      AS organizacion,
  m.variable,
  m.valor,
  m.timestamp
FROM mediciones m
JOIN dispositivos d ON m.dispositivo_id = d.dispositivo_id
JOIN ubicaciones u ON d.ubicacion_id = u.ubicacion_id
JOIN organizaciones o ON d.organizacion_id = o.organizacion_id

WHERE m.variable = $1
      AND m.timestamp BETWEEN $2 AND $3
ORDER BY m.timestamp DESC
LIMIT 100;
```

Checklist

- ✓ Seleccionar solo las columnas necesarias
- ✓ Usar nombres y alias comprensibles
- ✓ Verificar filtros antes de modificar datos
- ✓ Seguir las claves para los JOIN
- ✓ Pensar en el significado de cada grupo
- ✓ Formatear consultas complejas
- ✓ Dividir problemas en etapas
- ✓ Utilizar parámetros en aplicaciones



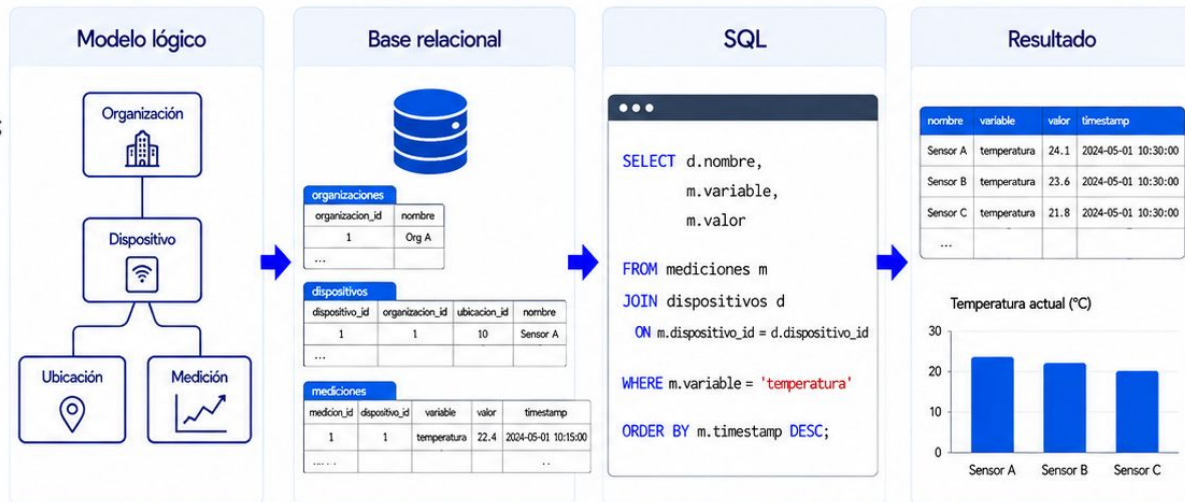
Una buena consulta debería ser correcta, comprensible y fácil de verificar.

¿Qué observamos?

Del modelo a información utilizable

Durante la clase:

- implementamos el modelo mediante tablas;
- convertimos identificadores y relaciones en PK y FK;
- incorporamos restricciones;
- creamos y modificamos datos;
- filtramos y ordenamos información;
- calculamos métricas;
- combinamos tablas;
- construimos consultas más complejas;
- utilizamos operaciones controladas.



El modelo define la estructura.
SQL nos permite utilizarla.

Resumen de la clase

Ideas principales

- El modelo relacional organiza información mediante tablas relacionadas.
- Las claves primarias identifican filas.
- Las claves foráneas implementan relaciones.
- Las restricciones ayudan a mantener datos válidos.
- DDL permite definir estructuras.
- DML permite insertar, modificar y eliminar datos.
- SELECT permite recuperar información.
- WHERE y ORDER BY permiten controlar qué datos obtenemos y cómo se presentan.
- Las agregaciones permiten construir métricas.
- JOIN combina información relacionada.
- Subconsultas, CTE y funciones de ventana permiten expresar consultas más complejas.
- Las transacciones agrupan operaciones.
- Las consultas parametrizadas separan instrucciones y valores.



Una base relacional no es solamente un conjunto de tablas: es una estructura que podemos consultar y operar manteniendo relaciones y reglas.

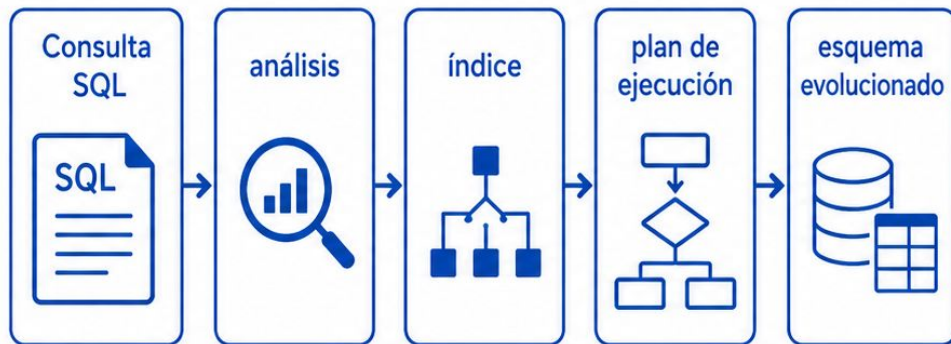
Próxima clase

Diseño Relacional, Optimización y Evolución del Esquema

Ya podemos crear y consultar una base relacional.

En la próxima clase vamos a preguntarnos:

- ¿la estructura está correctamente diseñada?;
- ¿hay información redundante?;
- ¿cómo normalizamos el modelo?;
- ¿cómo aceleramos consultas frecuentes?;
- ¿qué índice conviene utilizar?;
- ¿cómo sabemos qué está haciendo PostgreSQL?;
- ¿cuándo conviene utilizar JSON?;
- ¿cómo evoluciona el esquema cuando cambian los requerimientos?



Hoy aprendimos a utilizar el modelo.

La próxima clase veremos cómo mejorar su diseño y su rendimiento.



Enlace a la encuesta



Muchas gracias ¿Dudas?